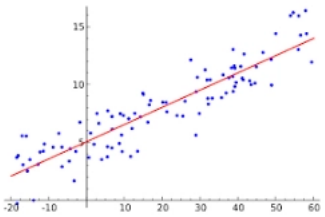
6. Supervised Machine Learning

Linear regression and classification models build on a single idea: using mathematical functions to trace the path through real-world data points with as little error as possible.

1. Linear Regression



Linear regression finds the best possible straight line (or flat plane) through a set of data points to predict an unknown continuous number.

At its core, this is simply **curve fitting** using a known mathematical template:

$$\hat{y} = w \cdot x + b$$

- x **(Input Feature):** The information you feed the model (e.g., house size in square feet).
- $\hat{y}$ **(Prediction):** The number the model outputs (e.g., estimated house price).
- w **(Weight):** The slope of the line. It controls how steep the line is and reflects how strongly the feature x affects the prediction.
- b **(Bias):** The offset or vertical intercept. It shifts the line up or down so it doesn't have to pass through $(0, 0)$. It acts as a baseline value when $x = 0$.

Fitting the curve simply means sliding and tilting the line by testing different values of w and b until it sits as close to every actual data point as possible.

2. The Error Function: Sum of Squared Errors (SSE)

To know whether a line fits well, the computer needs a score sheet. That score sheet is called an **error function** (or loss function).

For every data point i:

1. The real value is y_i.
2. The model's prediction is $\hat{y}_i$.
3. The gap between them is the error: $e_i = y_i - \hat{y}_i$.

The **Sum of Squared Errors (SSE)** squares each individual mistake and adds them all together:

$$\text{SSE} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{n} \Big(y_i - (w \cdot x_i + b) \Big)^2$$

Why square the errors?

- **Cancels signs:** If one point is 5 units above the line and another is 5 units below, simply adding them would give zero error (+5 - 5 = 0). Squaring turns all errors positive (25 + 25 = 50).
- **Penalizes big mistakes:** Squaring turns an error of 2 into 4, but an error of 10 into 100. This forces the algorithm to avoid extreme misses.

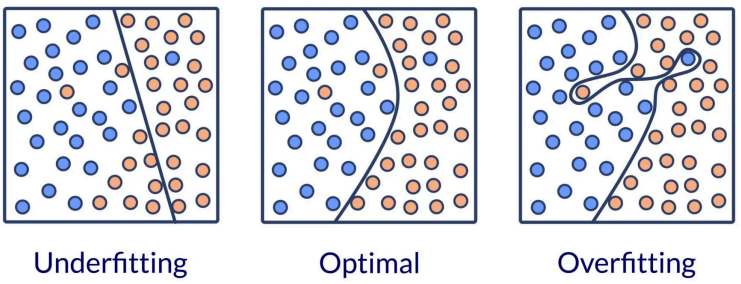
Why SSE is a function of weights and biases

In any training dataset, the actual data points (x_i, y_i) are permanent facts; they never change. The only variables you can adjust are the slope w and the intercept b.

Because of this, $\text{SSE}(w, b)$ forms a bowl-shaped surface. Finding the "best fit" line means finding the specific pair of (w, b) values that sits at the very bottom of that bowl, where error is lowest.

3. Underfitting (High Bias) vs. Overfitting (High Variance)

When fitting a curve through data, models struggle with two common failure modes:



Underfitting (High Bias)

- **What happens:** The model is far too simple to capture the underlying pattern (e.g., forcing a rigid straight line onto data that naturally curves like an S).
- **The meaning of "Bias":** The algorithm brings rigid, false assumptions to the table and ignores what the data is actually trying to show.
- **Result:** High error on training data, and high error on new, unseen test data.

Overfitting (High Variance)

- **What happens:** The model is overly complex and flexible (e.g., using a high-degree polynomial that zigzags to touch every single dot). It memorizes background noise and random flukes instead of the true pattern.
- **The meaning of "Variance":** If you trained this model on a slightly different sample of data, the fitted curve would swing wildly and look completely different.
- **Result:** Near-zero error on training data, but huge error on new test data because it cannot generalize.

The Sweet Spot (Balanced Fit)

A good model captures the broad underlying shape of the data while smoothly ignoring random noise and outliers.

4. How Training Works: Learning Rate and Backpropagation

Training is the automated process of rolling down the SSE error bowl to reach the lowest point (the minimum error).

Gradient Descent & Learning Rate

1. The model starts with random guesses for w and b.
2. It checks the slope of the error bowl at its current position (the *gradient*), which points downhill toward lower error.
3. It takes a small step in the downhill direction to update w and b.

The size of that step is controlled by the **Learning Rate (α)**:

- **Too large:** The model takes giant leaps, jumps back and forth across the valley, misses the bottom entirely, and may spiral out of control.
- **Too small:** The model takes microscopic steps. Training takes hours or days, and it risks stalling before reaching the bottom.

5. Logistic Regression: Classification via Probabilities

Despite having the word "regression" in its name, **logistic regression is a classification algorithm**. It is used to predict the likelihood that an input belongs to a specific category—most commonly a binary choice such as **Yes/No**, **Spam/Not Spam**, or **Pass/Fail**.

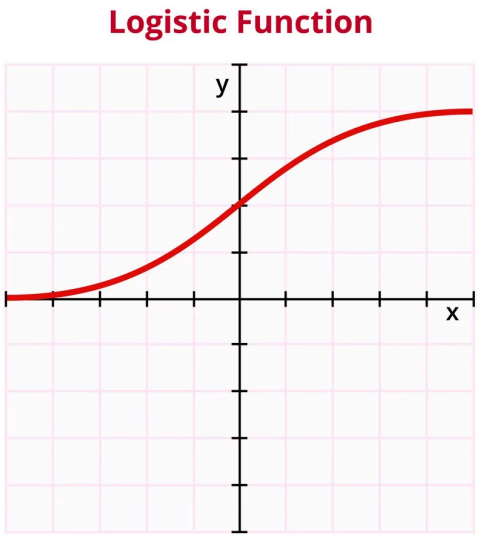
1. Why Not Just Use a Straight Line?

If you try to classify data using linear regression ($y = w \cdot x + b$), you run into two major issues:

- **Unbounded Outputs:** A straight line continues forever in both directions. It can output values like -1.5 or $+3.2$. But a probability must always stay strictly between **0 (0%)** and **1 (100%)**.

- **Sensitivity to Outliers:** Adding a single extreme data point far to the right or left tilts the entire straight line, shifting the cutoff point and causing correct classifications to become wrong.

2. The Sigmoid (Logistic) Function



To solve this, logistic regression calculates the standard linear combination first:

$$z = w \cdot x + b$$

Then, instead of outputting z directly, it feeds z into a special mathematical curve called the **Sigmoid function** (denoted as \sigma):

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

This function acts like a numerical clamp:

- **Large positive** z : e^{-z} approaches 0 , so $\sigma(z)$ approaches **1 (100%)**.
- **Large negative** z : e^{-z} grows massive, so the denominator explodes and \sigma(z) approaches **0 (0%)**.
- **When** $z = 0 : \sigma(0) = \frac{1}{1+1} = \mathbf{0.5}$ **(50%)**.

No matter how large or small the input number is, the Sigmoid function squashes it into a smooth, S-shaped curve strictly contained between 0 and 1 .

3. Turning Probabilities into Decisions (The Decision Boundary)

Once the model calculates a probability, it uses a **decision threshold** (by default, **0.5**) to make a final categorical call:

$$\text{Prediction} = \begin{cases} 1 & \text{if } \hat{y} \geq 0.5 \\ 0 & \text{if } \hat{y} < 0.5 \end{cases}$$

- If the model estimates an email has a **0.82 (82%)** chance of being spam, it classifies it as **Spam (1)**.
- If the estimate is **0.15 (15%)**, it classifies it as **Inbox (0)**.

The exact line or surface where z = 0 (and probability equals 0.5) is called the **decision boundary**. Data on one side of this boundary gets labeled Class 1, and data on the other side gets labeled Class 0.

6. Linear Regression vs. Logistic Regression

Despite sharing the word "regression" in their names, these two algorithms serve fundamentally different goals.

Feature	Linear Regression	Logistic Regression
Primary Task	**Regression** (predicting continuous amounts)	**Classification** (predicting categories / labels)
Target Output	Any continuous number ($-\infty$ to $+\infty$)	A probability score strictly bounded between **0 and 1**
Core Equation	$\hat{y} = w \cdot x + b$	$\hat{y} = \frac{1}{1+e^{-(w \cdot x + b)}}$ (Sigmoid function)
Typical Use Case	Predicting house prices, stock values, or outdoor temperature	Predicting whether an email is spam (0 or 1), or a tumor is malignant